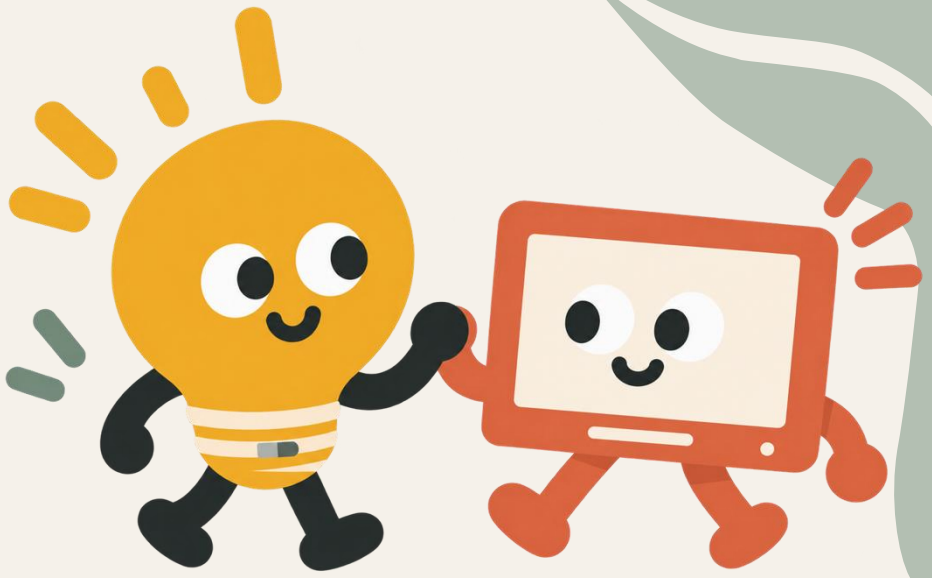
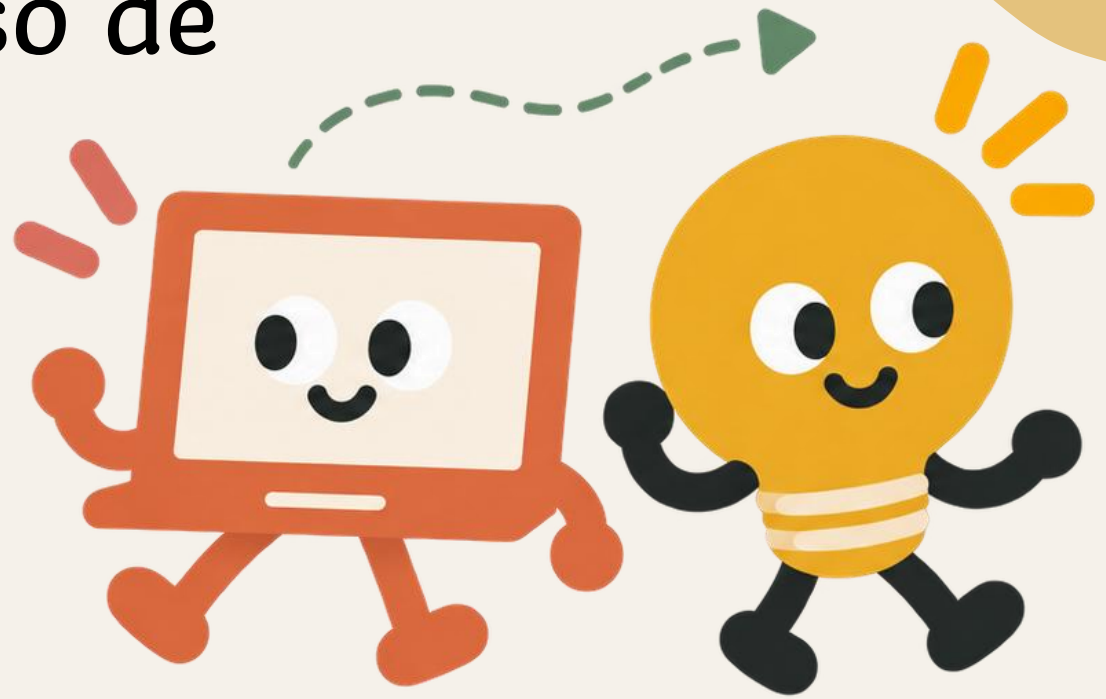
Módulo 6

Taller teórico-práctico
Del dataset a U-Net: segmentación de imágenes con Deep Learning

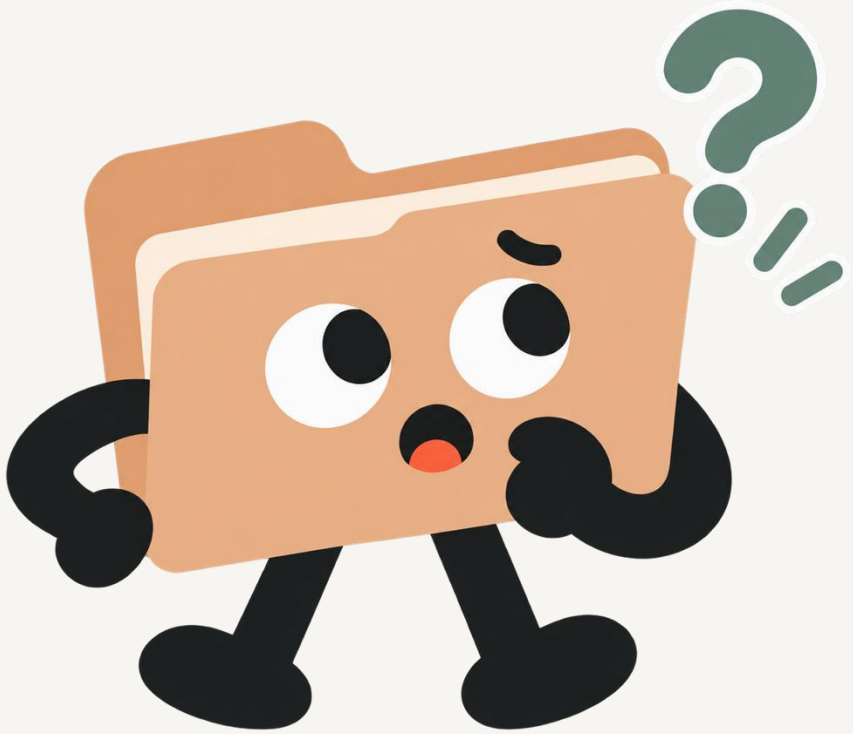
→ Entrenamiento, evaluación y análisis



Fundamentos y proceso de entrenamiento



¿Cómo aprende U-Net a producir máscaras cada vez más parecidas a las anotaciones reales y qué operaciones permiten modificar sus parámetros para reducir los errores?



Entrenamiento de una red neuronal

Batch, iteración, época y forward pass: conceptos clave para entender cómo se entrena nuestra Mini U-Net.

1 Batch

¿Cuántas imágenes procesamos juntas?

Conjunto de muestras que se introducen simultáneamente en el modelo.



$X \in \mathbb{R}^{16\times3\times256\times256}$
(batch_size = 16)



$\hat{Y} \in \mathbb{R}^{16\times1\times256\times256}$

2 Iteración

¿Qué sucede al procesar un batch?

Es el ciclo de entrenamiento que procesa un batch y (generalmente) actualiza los parámetros.

- **Forward pass**
  Pronósticos del modelo (logits).
- **Cálculo de la pérdida**
  Compara predicciones y máscaras reales.
- **Backward pass**
  Calcula los gradientes (backpropagation).
- **Actualización de pesos**
  El optimizador modifica los parámetros.

3 Epoch (época)

¿Qué significa recorrer el conjunto de entrenamiento?

- Un recorrido completo de todas las imágenes de entrenamiento.
- Comprende todos los batches.
- Las mismas imágenes se procesan nuevamente (puede cambiar su orden si shuffle=True).

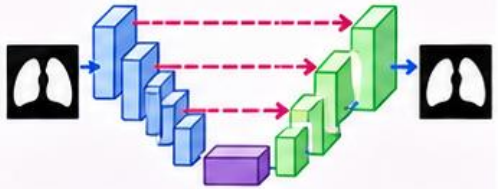
Ejemplo:

100 imágenes
batch_size = 16
Batches por época: **7**
Épocas: **9**

4 Forward pass

¿Qué ocurre dentro del modelo?

Es el cálculo de la salida de la red a partir de una entrada, con los parámetros actuales (no actualiza los pesos).



1. Extraer características (encoder).
2. Procesarlas en el bottleneck.
3. Reconstruir la resolución (decoder).
4. Combinar con skip connections.
5. Producir logits con conv. de 1 × 1.

Ejemplo completo

100 imágenes
batch_size = 16
épocas = 9

División en batches (por época)

B1 16	B2 16	B3 16	B4 16
B5 16	B6 16	B7 4	

Batches por época:

$N_{batches} = \lceil 100/16 \rceil = 7$

Iteraciones totales:

$9 \times 7 = 63$

(= 63 actualizaciones, sin acumulación de gradientes)

Ciclo de entrenamiento

Época 1 (7 iteraciones)

B1 B2 B3 B4 B5 B6 B7

...

Época 9 (7 iteraciones)

B1 B2 B3 B4 B5 B6 B7

Concepto	Pregunta que responde
Batch	¿Cuántas imágenes procesamos juntas?
Iteración	¿Cuántas veces ejecutamos el ciclo de procesamiento de un batch (forward, pérdida, backward y actualización)?
Epoch	¿Cuántas veces recorremos todo el conjunto de entrenamiento?
Forward pass	¿Cómo calcula la red las predicciones a partir de las imágenes?



En síntesis

- El batch es un grupo de imágenes.
- Una iteración procesa un batch y actualiza el modelo.
- Una época comprende todos los batches.
- El forward pass calcula las predicciones (logits) a partir de la entrada.
- En el ejemplo: 100 imágenes, batch_size = 16, 7 batches por época y 9 épocas → 63 iteraciones (y 63 actualizaciones).

Ciclo de entrenamiento: pasos y conceptos

Proceso completo para entrenar la Mini U-Net usando batches, iteraciones, épocas y el forward pass.

Contexto del ejemplo



100 imágenes de entrenamiento



batch_size = 16



épocas = 9

Cálculos

- **Batches por época:** $N_{batches} = [100/16] = 7$
- **Iteraciones totales:** $9 \times 7 = 63$
- **Por época:** 6 batches de 16 imágenes y 1 de 4.

¿Qué ocurre en una iteración? (procesamiento de un *batch*)

1 Reiniciar gradientes

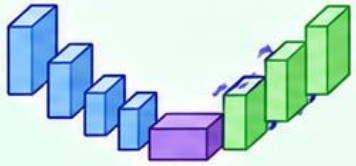
Limpia los gradientes acumulados.



optimizador.zero_grad()

2 Forward pass (paso hacia adelante)

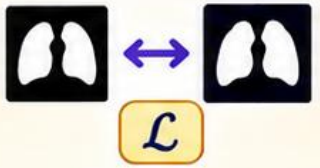
Calcula la salida del modelo (logits) a partir del batch de imágenes.



logits = modelo(imagenes)

3 Cálculo de la pérdida

Compara las predicciones con las máscaras reales (p. ej., BCEWithLogitsLoss).



loss = criterio(logits, mascaras)

4 Backward pass (retropropagación)

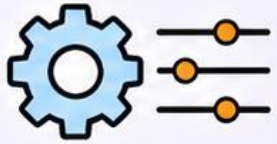
Calcula los gradientes de la pérdida respecto a los parámetros.



loss.backward()

5 Actualización de pesos

El optimizador modifica los parámetros del modelo (p. ej., Adam).



optimizador.step()

Esta secuencia constituye **UNA ITERACIÓN** (un batch).

Ciclo completo del entrenamiento

Época 1 (7 iteraciones)

B1	B2	B3	B4	B5	B6	B7
16	16	16	16	16	16	4

...

Época 9 (7 iteraciones)

B1	B2	B3	B4	B5	B6	B7
16	16	16	16	16	16	4

9 épocas = **63 iteraciones** (y 63 actualizaciones, sin acumulación de gradientes).

En síntesis



- **Batch:** grupo de imágenes que se procesa junto (16, excepto el último con 4).
- **Iteración:** ejecución de los pasos 1–5 para un batch (forward, pérdida, backward y actualización).
- **Epoch (época):** recorrido completo del conjunto de entrenamiento (7 batches = 100 imágenes).
- **Forward pass:** cálculo de la salida del modelo (logits) a partir de las imágenes, sin actualizar los pesos.

Función de pérdida vs. Métrica

Ambas cuantifican qué tan bien predice el modelo, pero cumplen roles distintos en el entrenamiento.

Contexto: segmentación

La Mini U-Net recibe una imagen y genera una máscara binaria.



X (imagen)

Y (máscara real)



$\hat{P}$ (probabilidades)



- Y: valores 0 y 1.
- $\hat{P}$: valores en [0, 1].

Función de pérdida (*loss*)

- Cuantifica la diferencia entre la predicción y la máscara real.
- Guía la optimización de los parámetros del modelo.
- Se minimiza durante el entrenamiento.

$$\theta^* = \arg\min_{\theta} \mathcal{L}\left(f_\theta(X), Y\right)$$

En PyTorch (entrenamiento)

```
logits = modelo(imagenes)
loss = criterio(logits, mascaras)
loss.backward()  # colcula gradientes
optimizador.step() # actualiza pesos
```

Métrica de evaluación

- Mide el rendimiento del modelo.
- Se usa para interpretar y comparar resultados.
- Normalmente no se utiliza para actualizar los pesos.

Dice Score
Solapamiento entre máscaras.

IoU
Intersección sobre la unión.

Precision
Proporción de predicciones positivas correctas.

Recall
Proporción de positivos reales detectados.

Principales diferencias

Característica	Función de pérdida (loss)	Métrica
Propósito	Optimizar el modelo	Evaluar su rendimiento
Uso de gradientes	Habitualmente sí	Habitualmente no
Se utiliza en backward()	Sí	Generalmente no
¿Se minimiza o maximiza?	Generalmente se minimiza	Depende de la métrica (p. ej., Dice, IoU se maximizan)
Ejemplo	Dice Loss	Dice Score

¿Puede una misma formulación utilizarse para ambas?

Sí. El coeficiente Dice es un ejemplo claro.

$$Dice = \frac{2|A \cap B|}{|A| + |B|}$$

Predicción (umbral ≥ 0.5)



A

Máscara real



B

$$Dice\ Score = \frac{2|A \cap B|}{|A| + |B|} = \mathbf{0.92} \rightarrow \underline{Dice}\ Loss = 1 - Dice = 1 - 0.92 = \mathbf{0.08}$$

- Durante el entrenamiento se usa una versión diferenciable (soft Dice) con probabilidades.
- Durante la evaluación se calcula Dice Score sobre máscaras binarias (con un umbral).

En síntesis



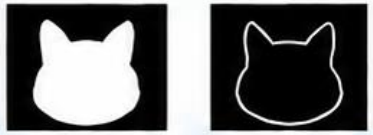
- La **función de pérdida (*loss*)** cuantifica el error y se minimiza para entrenar el modelo.
- La **métrica** mide el desempeño y sirve para interpretar y comparar resultados.
- Pueden estar relacionadas (p. ej., Dice Score y Dice Loss), pero no siempre producen el mismo valor.

Funciones de pérdida para segmentación

Cuantifican el error entre la predicción del modelo y la máscara real, y se utilizan para entrenar la red.

1 Binary Cross-Entropy (BCE)

Mide qué tan bien coinciden las probabilidades predichas con las etiquetas reales (0: fondo, 1: objeto) para cada píxel.



y = 1 (objeto)

y = 0 (fondo)

$$\mathcal{L}_{BCE}(p, y) = -[y \ln(p) + (1 - y) \ln(1 - p)]$$

Pixel	y	p (predicha)	BCE
A	1	0.95	0.051
B	1	0.60	0.511
C	1	0.10	2.303

La pérdida aumenta cuando el modelo asigna una probabilidad baja a la clase correcta.

BCE (y = 1)



La BCE penaliza especialmente las predicciones equivocadas con mucha confianza.

BCEWithLogitsLoss en PyTorch

Combina internamente Sigmoid + BCE y permite trabajar directamente con los logits.

```
criterio = nn.BCEWithLogitsLoss()
logits = modelo(imagenes)
loss = criterio(logits, mascaras.float())
```

logits (z) → Sigmoid $\sigma(z)$ → BCE $\mathcal{L}_{BCE}$

Ventajas

- Señal de error por píxel.
- Sencilla y efectiva.
- Buen punto de partida.

Limitaciones

- Puede estar influida por el predominio del fondo. (objetos pequeños).

pos_weight

Permite dar mayor importancia a los pixeles positivos.

```
nn.RCEWithLogitsLoss(
    pos_weight=torch.tensor([3.01]))
```

2 Dice Loss

Se basa en el coeficiente Dice, que mide el solapamiento entre la máscara predicha y la real.



Máscara real (A)



Máscara predicha (B)

$$Dice(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

$$\mathcal{L}_{DICE} = 1 - \frac{2 \sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + e}$$

- p_i : probabilidad predicha del pixel i.
- y_i : etiqueta real (0 o 1).
- ϵ : constante pequeña (estabilidad).

Ejemplo (máscaras binarias)



$|A| = 20$ $|B| = 20$ $|A \cap B| = 15$

$$Dice = \frac{2(15)}{20 + 20} = 0.75$$

$$\mathcal{L}_{DICE} = 1 - 0.75 = 0.25$$

Implementación con MONAI

Utiliza una versión diferenciable de Dice y aplica Sigmoid internamente (sobre los logits).

```
from monai.losses import DiceLoss
criterio = DiceLoss(sigmoid=True)
loss = criterio(logits, mascaras.float())
```

logits (z) → Sigmoid $\sigma(z)$ → Dice $\mathcal{L}_{DICE}$

Ventaja

- Considera el solapamiento global de la región, útil cuando el objeto es pequeño.

Limitación

- Su comportamiento puede depender del suavizado (e) y de cómo se agregan las muestras, especialmente en regiones vacías o muy pequeñas.

En síntesis



- **BCE:** mide el error por pixel y es un buen punto de partida; se recomienda usar nn.BCEWithLogitsLoss().
- **Dice Loss:** se basa en el solapamiento entre máscaras y es útil con objetos pequeños.
- **Ambas:** se utilizan con logits durante el entrenamiento y guían la optimización de los parámetros del modelo.

Focal Loss y Generalized Dice Focal Loss

Funciones de pérdida diseñadas para abordar píxeles difíciles y el desequilibrio entre clases en segmentación.

4 Focal Loss

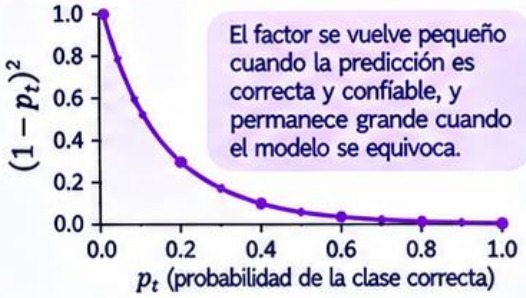
Reduce la influencia de los ejemplos fáciles y concentra el aprendizaje en los ejemplos que el modelo todavía clasifica incorrectamente.

$$\mathcal{L}_{Focal} = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

- p_t : probabilidad de la clase correcta.
- γ : controla cuánto disminuye la contribución de los ejemplos fáciles.
- α_t : peso opcional de la clase.

Factor de modulación: $(1 - p_t)^{\gamma}$

Ejemplo ($\gamma = 2$)	p_t	$(1 - p_t)^2$
Fácil	0.95	0.0025
Intermedio	0.60	0.16
Difícil	0.10	0.81



El factor se vuelve pequeño cuando la predicción es correcta y confiable, y permanece grande cuando el modelo se equivoca.

Implementación con MONAI

```
from monai.losses inport FocalLoss
criterio = FocalLoss(gamma=2.0)
loss = criterio(logits, mascaras.float())
```

¿Cuándo es útil?

- Cuando hay muchos píxeles fáciles (p. ej., grandes regiones de fondo) y relativamente pocos difíciles.
- Se centra en los errores difíciles, pero no reemplaza el balance entre clases.

5 Generalized Dice Focal Loss (GDFL)

Combina dos objetivos complementarios:

- **Generalized Dice Loss (GDL):** evalúa el solapamiento y utiliza ponderaciones según el tamaño de las clases.
- **Focal Loss:** reduce la contribución de las predicciones fáciles.

$$\mathcal{L}_{GDFL} = \lambda_{gdl}\mathcal{L}_{GDL} + \lambda_{focal}\mathcal{L}_{Focal}$$

Generalized Dice Loss

$$GDL = 1 - \frac{2\sum_c w_c \sum_i p_{ic} y_{ic} + \epsilon}{\sum_c w_c \sum_i (p_{ic} + y_{ic}) + \epsilon} \qquad w_c = \frac{1}{\left(\sum_i y_{ic}\right)^2}$$

- p_{ic} : probabilidad del píxel i para la clase c.
- y_{ic} : etiqueta real del píxel i para la clase c.
- w_c : peso de la clase (p. ej., inverso del cuadrado de su tamaño).
- ϵ : constante pequeña (estabilidad numérica).



Fondo (95 %)



Objeto (5 %)

Ejemplo: imagen con fuerte desequilibrio de clases.

- **GDL:** equilibra por tamaño.
- **Focal:** se enfoca en errores difíciles.

Implementación con MONAI

```
from monai.losses import
GeneralizedDiceFocalLoss
criterio = GeneralizedDiceFocalLoss(
    sigomid=True,
    gamma=2.0,
    lambda_gdl=1.0,
    lambda_focal=1.0)
loss = criterio(logits, mascaras.float())
```

¿Por qué combinarla?

- GDL equilibra la contribución de las clases (útil cuando el objeto es pequeño).
- Focal Loss pone mayor énfasis en los píxeles difíciles.

Precisión para Mini U-Net

Con una sola salida (N, 1, H, W), la ventaja de ponderar ambas clases en GDL no se manifiesta igual que en una representación de dos canales.

Comparación de las pérdidas



Función de pérdida	¿Qué penaliza principalmente?	Ventaja	Consideración
BCE	Errores de probabilidad por píxel.	Simple, estable y fácil de interpretar.	Puede estar dominada por el fondo.
Dice Loss	Falta de solapamiento entre regiones.	Optimiza directamiente una aproximación al solapamiento.	Sensible a regiones vacías y al suavizado.
Focal Loss	Predicciones difíciles, reduciendo el peso de las fáciles.	Útil cuando abundan píxeles fáciles.	Requiere elegir γ y, opcionalmente, pesos.
Generalized Dice Focal Loss	Solapamiento ponderado y predicciones difíciles.	Combina dos objetivos complementarios.	Más hiperparámetros y mayor complejidad.

Backpropagation y entrenamiento

La backpropagation (retropropagación) calcula cómo contribuye cada parámetro al error de las predicciones. No actualiza los pesos: eso lo hace el optimizador.

Ciclo de entrenamiento (por batch)



(X, Y)
imágenes y máscaras reales

1 Forward pass

La red calcula las predicciones.



$\hat{Y}$
(logits)

2 Loss

Se calcula el error respecto a las máscaras reales.

$$\mathcal{L}(\hat{Y}, Y)$$

3 Backward pass

Se calculan los gradientes de los parámetros (backpropagation).

$$\frac{\partial \mathcal{L}}{\partial w}$$

4 Optimizer step

Se actualizan los pesos para intentar reducir la pérdida.

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

¿Qué es un gradiente?

Indica cómo cambia la pérdida cuando modificamos un parámetro. $\frac{\partial \mathcal{L}}{\partial w}$

- ↑ **Gradiente positivo:** aumentar w tiende a aumentar la pérdida (conviene disminuirlo).
- ↓ **Gradiente negativo:** aumentar w tiende a reducir la pérdida.
- ≈ **Cercano a cero:** cambios pequeños en w tienen poco efecto.

Acumulación de gradientes

En PyTorch, cada `loss.backward()` suma los nuevos gradientes a los ya almacenados en `.grad`.

Estado inicial	1.º backward()	2.º backward()	3.º backward()
w.grad = 0	0 + 0.4 = 0.4	0.4 + 0.3 = 0.7	0.7 + 0.2 = 0.9

✓ Es útil para acumular gradientes de varios batches (p. ej., para simular un batch más grande) antes de actualizar los pesos.

Comandos clave en PyTorch

`optimizer.zero_grad()`	Reinicia los gradientes acumulados (antes del backward()).
`modelo(imagenes)`	Calcula las predicciones (forward pass).
`loss = criterio(...)`	Calcula la pérdida.
`loss.backward()`	Calcula y acumula gradientes (backpropagation).
`optimizer.step()`	Actualiza los parámetros usando esos gradientes.

Idea clave



La **backpropagation** calcula los gradientes (cómo deberían cambiar los pesos), y el **optimizador** utiliza esa información para actualizarlos.

¿Cuándo usar optimizer.zero_grad()?

loss.backward() acumula gradientes. optimizer.zero_grad() indica que comenzamos una nueva acumulación.



Actualizar por cada batch

Cada batch produce y utiliza sus propios gradientes.

✓ Con optimizer.zero_grad() (correcto)

Batch 1
optimizer.zero_grad()
loss.backward() → **0.4**
optimizer.step() (usa **0.4**)

↓

Batch 2
optimizer.zero_grad()
loss.backward() → **0.3**
optimizer.step() (usa **0.3**)

✗ Sin optimizer.zero_grad() (acumulación no deseada)

Batch 1
loss.backward() → **0.4**
optimizer.step()
(usa **0.4**)

↓

Batch 2
loss.backward() → **+0.3**
(gradiente total: **0.7**)
optimizer.step()
(usa **0.7**)

```
# Actualización por cada batch
for imagenes, mascaras in train_loader:
    optimizer.zero_grad()
    logits = modelo(imagenes)
    loss = criterio(logits, mascaras)
    loss.backward()
    optimizer.step()
```



Acumular gradientes (simular un batch grande)

Útil si no podemos procesar todas las imágenes a la vez (p. ej., batch efectivo de 16 = 4 microbatches de 4).

Microbatch 1 (4 imágenes)	Microbatch 2 (4 imágenes)	Microbatch 3 (4 imágenes)	Microbatch 4 (4 imágenes)
g = 0.4	g = 0.3	g = 0.2	g = 0.5

Gradiente promedio: $g = \frac{0.4 + 0.3 + 0.2 + 0.5}{4} = 0.35$

```
# Acumulación de 4 microbatches
acumulación = 4
optimizer.zero_grad()
for i, (imagenes, mascaras) in enumerate(train_loader):
    loss = criterio(modelo(imagenes), mascaras) / acumulacion
    loss.backward()          # acumula gradientes
    if (i + 1) % acumulacion == 0:
        optimizer.step()      # una sola actualización
        optimizer.zero_grad()
```

Momentos clave de optimizer.zero_grad()

✓ **Antes del primer** loss.backward()	**Correcto:** comenzamos una nueva acumulación.
✗ **Entre microbatches** que queremos acumular	**Incorrecto:** perdemos los gradientes previos.
✗ **Después de** loss.backward() y antes de optimizer.step()	**Incorrecto:** eliminamos los gradientes que íbamos a usar.
✓ **Después de** optimizer.step()	**Correcto:** preparamos la siguiente acumulación.

Idea clave

Acumular gradientes es reunir información de varios microbatches antes de actualizar los pesos.

Regla práctica

- **Actualización por batch:** usa optimizer.zero_grad() en cada iteración.
- **Acumulación de gradientes:** limpia al comenzar la acumulación, ejecuta varios loss.backward() y actualiza una sola vez (al final del grupo).

Optimizadores en el entrenamiento

Actualizan los parámetros de la red utilizando los gradientes calculados por backpropagation, con el objetivo de reducir la pérdida.

¿Cómo se actualizan los pesos?

$$w_{t+1} = w_t - \eta \frac{\partial \mathcal{L}}{\partial w_t}$$

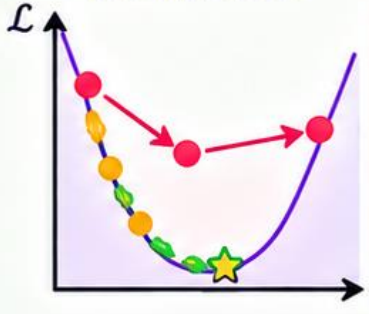
- w_t : peso actual.
- η : learning rate (tasa de aprendizaje).
- $\frac{\partial \mathcal{L}}{\partial w_t}$: gradiente de la pérdida.

Ejemplo: $w_t = 0.8 \quad \eta = 0.1 \quad \frac{\partial \mathcal{L}}{\partial w_t} = 0.4$

$w_{t+1} = 0.8 - (0.1)(0.4) =$ **0.76**

Learning rate (η)

Controla el tamaño del paso de actualización.



- **η muy alto:** puede oscilar o divergir.
- **η muy bajo:** avanza muy lento.
- **η adecuado:** progreso estable.

Optimizadores más utilizados

SGD
- Descenso de gradiente directo.
- Puede usar momentum (usa la dirección previa).



Adam
- Adaptativo: utiliza primer y segundo momento de los gradientes.
- Buen punto de partida.



AdamW
- Adam con weight decay desacoplado (decaimiento de pesos).
- Ayuda a regularizar el modelo.



Comparación: SGD vs. Adam vs. AdamW

Característica	SGD	Adam	AdamW
Actualización	Gradiente directo	Adaptativa	Adaptativa
Momentos	Opcional (momentum)	Primer y segundo momento	Primer y segundo momento
LR adaptativo por parámetro	✗ No	✓ Sí	✓ Sí
Weight decay desacoplado	✗ No	✗ No	✓ Sí
Uso frecuente	Con ajustes cuidadosos	Punto de partida general	Entrenamiento con regularización

Ciclo de entrenamiento

1.  **optimizer.zero_grad()** Reinicia los gradientes.
2.  **logits = modelo(imagenes)** Forward pass: predicciones.
3.  **loss = criterio(...)** Calcula la pérdida.
4.  **loss.backward()** Calcula los gradientes (backpropagation).
5.  **optimizer.step()** Actualiza los pesos.

Ejemplo: con Adam

```
optimizer = torch.optim.Adam(
    modelo.parameters(),
    lr=1e-3)

for imagenes, mascaras in train_loader:
  optimizer.zero_grad()
  logits = modelo(imagenes)
  loss = criterio(logits, mascaras.float())
  loss.backward()
  optimizer.step()
```



Idea clave

El **gradiente** indica cómo cambia la pérdida respecto a cada parámetro; el **learning rate** (η) controla la magnitud de las actualizaciones; y el **optimizador** (SGD, Adam o AdamW) define la regla para actualizar los pesos.